

Documentación del Algoritmo FFDLS (First Fit Decreasing using Largest bins, with Shifting)

EQUIPO 3

April 24, 2025

1 Introducción

El problema de *Variable Sized Bin Packing* (VSBP) es un problema de optimización combinatoria, donde el objetivo es empaquetar un conjunto de elementos de tamaños variados en bins de capacidades también variadas, de tal forma que se minimice el número total de bins utilizados, o el espacio desperdiciado dentro de estos bins. Este problema es NP-duro, lo que implica que no existen algoritmos exactos eficientes para resolverlo en tiempo polinómico.

Las heurísticas son una aproximación eficiente para resolver problemas NP-duros. En este trabajo, presentamos el algoritmo **FFDLS** (First Fit Decreasing using Largest bins, with Shifting), que es una mejora del algoritmo **First Fit Decreasing** (FFD) clásico, con un paso adicional de *shifting* para mejorar el empaquetado y reducir el espacio desperdiciado.

2 Planteamiento del Problema

Dado un conjunto de elementos $P = \{p_1, p_2, \dots, p_n\}$ con tamaños p_i , y un conjunto de bins $B = \{b_1, b_2, \dots, b_k\}$ con capacidades b_j , el objetivo del problema de *Variable Sized Bin Packing* (VSBP) es empaquetar todos los elementos de manera que se minimice el número total de bins utilizados o el espacio desperdiciado en los bins. Las restricciones son las siguientes:

$$\sum_{i=1}^n p_i \leq \sum_{j=1}^k b_j$$

Cada bin b_j tiene una capacidad máxima, y los elementos deben ser colocados en bins de tal manera que la suma de los tamaños de los elementos en un bin no exceda la capacidad del bin.

La solución debe cumplir con las siguientes condiciones:

$$\sum_{i \in I_j} p_i \leq b_j \quad \text{para cada bin } b_j \in B$$

Donde I_j es el conjunto de elementos colocados en el bin b_j .

3 Modelo Matemático

El modelo matemático para el problema de bin packing de tamaño variable puede ser descrito utilizando una variable binaria x_{ij} , que indica si el elemento p_i es colocado en el bin b_j o no. La formulación del problema es la siguiente:

Minimizar:

$$\text{Costo Total} = \sum_{j=1}^k \delta_j$$

Donde $\delta_j = 1$ si el bin b_j está siendo utilizado, y 0 en caso contrario. El problema tiene las siguientes restricciones:

1. Cada elemento debe ser asignado a un bin:

$$\sum_{j=1}^k x_{ij} = 1, \quad \forall i \in P$$

2. La capacidad del bin no debe exceder la capacidad de su tamaño:

$$\sum_{i=1}^n p_i x_{ij} \leq b_j \delta_j, \quad \forall j \in B$$

3. La variable x_{ij} es binaria:

$$x_{ij} \in \{0, 1\}, \quad \forall i \in P, \forall j \in B$$

4. La variable δ_j es binaria:

$$\delta_j \in \{0, 1\}, \quad \forall j \in B$$

El objetivo es minimizar el número total de bins utilizados, o el espacio desperdiciado en los bins.

4 Justificación del Algoritmo FFDLS

El algoritmo FFDLS (First Fit Decreasing using Largest bins, with Shifting) es una mejora del algoritmo clásico **First Fit Decreasing** (FFD). En FFD, los elementos se colocan en el primer bin que pueda acomodarlos. FFDLS modifica este enfoque de la siguiente manera:

1. ****Ordenamiento Decreciente:**** Los elementos se ordenan de mayor a menor tamaño antes de ser empaquetados.
2. ****Shifting de Elementos:**** Si un bin tiene espacio desperdiciado, se intenta reubicar sus elementos en otros bins más pequeños que aún puedan acomodarlos, con el objetivo de reducir el desperdicio de espacio.

El algoritmo puede ser descrito en los siguientes pasos:

1. Se ordenan los elementos $p_1, p_2, \dots, p_n$ de mayor a menor tamaño.
2. Se selecciona un bin de la lista de bins disponibles y se coloca el primer elemento en él. Luego, se coloca el siguiente elemento en el primer bin disponible que tenga suficiente espacio.
3. Si el bin está parcialmente lleno y aún tiene espacio desperdiciado, se intenta realizar un *shifting* de las piezas dentro del bin a otro bin más pequeño, si es posible, para reducir el desperdicio.
4. Este proceso se repite hasta que todos los elementos hayan sido empaquetados.

La justificación detrás de este algoritmo se basa en dos puntos clave:

1. **Uso de los bins más grandes primero:** Esto reduce la posibilidad de que los bins más grandes queden parcialmente vacíos, lo que ayuda a minimizar el número total de bins utilizados.
2. **Shifting:** Al mover las piezas dentro de los bins, FFDLS intenta reducir el espacio desperdiciado en los bins parcialmente llenos.

4.1 Costo Computacional

El costo computacional del algoritmo FFDLS se compone de varias operaciones:

- **Ordenamiento de los elementos:** El ordenamiento de los elementos tiene un costo de $O(n \log n)$, donde n es el número de elementos.
- **Empaque de elementos:** Para cada elemento, buscamos el primer bin disponible, lo cual tiene un costo de $O(k)$ por cada elemento, donde k es el número de bins.
- **Shifting:** El *shifting* puede requerir la reubicación de elementos dentro de los bins, lo cual podría aumentar el costo en el peor de los casos a $O(n^2)$, si se realiza para cada elemento.

Por lo tanto, el costo total del algoritmo es de $O(n \log n + n \cdot k)$, lo que es eficiente en comparación con soluciones exactas.

4.2 Demostración de la Heurística

El algoritmo FFDLS ha demostrado ser eficiente debido a su capacidad para reducir el desperdicio de espacio dentro de los bins. A pesar de que no garantiza una solución óptima, la heurística tiene una cota de rendimiento de $\frac{4}{3}$ en el peor de los casos, lo que significa que el número de bins utilizados por el algoritmo no será más de $\frac{4}{3}$ veces el número de bins óptimos.

Esta cota de rendimiento proviene de la observación de que el algoritmo FFDLS realiza un empaque eficiente de los elementos, pero puede requerir una pequeña cantidad de bins adicionales debido a la falta de conocimiento completo sobre la disposición de los elementos.

5 Conclusiones

El algoritmo FFDLS presenta una mejora sobre el algoritmo clásico FFD al incorporar un paso de *shifting* para reducir el desperdicio de espacio. Aunque no garantiza una solución óptima, proporciona soluciones de alta calidad con una cota de rendimiento de $\frac{4}{3}$. Además, su costo computacional es razonable para su aplicación en escenarios prácticos.